

Module	Collaborative Development (5CS024)
Portfolio	1
Assessment Type	Individual Report

Doctor Appointment Booking System

Sprint 1 — Individual Developer Report

Sirjeet Dahal

Developer — Team Sigma

Student Number: 2462286

Submission Date: 18th April 2026

Acknowledgement

I would like to express my sincere gratitude to our module tutor for the continuous guidance, structured mentorship, and practical insight provided throughout Sprint 1 of this module. The instruction received on the principles of Agile Scrum, the ceremonies that underpin an effective sprint cycle, and the disciplined use of Jira as an issue-tracking and workflow-management platform significantly strengthened my practical understanding of modern collaborative software engineering.

I would also like to extend my appreciation to every member of Team Sigma for their cooperation, commitment, and professional dedication during the sprint. The shared accountability and willingness to participate in regular team meetings were instrumental in delivering the committed scope of the Doctor Appointment Booking System within the sprint window. Finally, I am grateful to Herald College for providing an environment in which Agile principles can be practised on a real, end-to-end software project rather than in isolation.

Abstract

This report presents a structured account of the individual contributions delivered during Sprint 1 of the Doctor Appointment Booking System project, developed by Team Sigma under the Collaborative Development module (5CS024). The project adopts an Agile Scrum methodology, with sprint backlogs, team meetings, sprint reviews, and retrospectives supported through Atlassian Jira. As a Developer assigned to full-stack feature delivery, my Sprint 1 commitment covered four user stories grouped under the Patient Dashboard Epic (SDABS-5): Home Page and Notifications Panel (SDABS-8), View Appointments (SDABS-6), Quick Booking Access (SDABS-7), and Profile Management (SDABS-9).

The technology stack adopted for the sprint comprised HTML and Tailwind CSS for the frontend, JavaScript for client-side interactivity, PHP for the REST backend, and MySQL for the relational data layer. Git was used for distributed version control and Jira for issue tracking and Agile sprint management. All four assigned tickets were completed and moved to Done status before the sprint review. Overall, Sprint 1 established a solid functional foundation for the patient-facing side of the Doctor Appointment Booking System and advanced my maturity as a professional software developer.

Table of Contents

Self-Appraisal Form	1
Personal Objectives — Performance Measurement.....	1
Collaboration Development.....	3
Good Communication and File Sharing	3
Continuing Personal Development (CPD).....	3
Issue Tracking	3
Work to Deadlines	3
Appendix A: Choosing Relevant Technologies.....	4
Technologies Used.....	4
Justification for Chosen Technologies	4
Risks and Limitations.....	4
Appendix B: Implementing Functional Requirements	6
Objective 1 — Home Page & Notifications Panel (SDABS-8)	6
Objective 2 — View Appointments (SDABS-6)	7
Objective 3 — Quick Booking Access (SDABS-7)	9
Objective 4 — Profile Management (SDABS-9).....	10
Use of Version Control	12
Appendix C: Evidence of Good Communication and File Sharing	15
Appendix D: Continuing Personal Development (CPD)	17
Appendix E: Evidence of Issue Tracking	19
Appendix F: Sprint Retrospective	20
What Went Well.....	20
Challenges Faced	21
Lessons Learned	21
Future Improvements for Sprint 2	21
Appendix G: Risk Register	22
Scoring Scheme	22
Risk Register	22
Priority Risks and Rationale.....	24
Review Cadence	25
References	26

Table of Figures

Figure 1 — Screenshot of the patient home page rendered in the browser (SDABS-8).....	6
Figure 2 — Screenshot of the notifications panel on the patient page (SDABS-8).....	7
Figure 3 — Screenshot of homepage.js in VS Code showing commented notification fetch and render logic (SDABS-8).....	7
Figure 4 — Screenshot of the View Appointments page in the browser (SDABS-6).....	8
Figure 5 — Screenshot of dashboard.js in VS Code showing commented appointment fetch and render logic (SDABS-6).....	9
Figure 6 — Screenshot of the Quick Booking Access button on the patient dashboard (SDABS-7).....	10
Figure 7 — Screenshot of booking.js in VS Code showing loadDoctors() function fetching available doctors from the API code (SDABS-7)	10
Figure 8 — Screenshot of the Profile Management page in the browser (SDABS-9)	11
Figure 9 — Screenshot of profile.js in VS Code showing commented profile fetch and update logic (SDABS-9).....	12
Figure 10 — Screenshot of GitHub commit history showing Sprint 1 commits by myself (page 1)	13
Figure 11 — Screenshot of GitHub commit history showing Sprint 1 commits by sirjit2 (page 2)	13
Figure 12 — Screenshot of GitHub contributor graph showing sirjit2 commit activity during Sprint 1	14
Figure 13 — Team Sigma in-person meeting during Sprint 1	15
Figure 14 — Jira Sprint 1 completion report for Team Sigma (SDABS board).....	15
Figure 15 — GitHub repository showing Team Sigma project with branches, commits, and contributors	16
Figure 16 — Traversy Media YouTube tutorial used during Sprint 1 for HTML, CSS and JavaScript reference	18
Figure 17 — The Net Ninja YouTube tutorial used during Sprint 1 for PHP and MySQL reference.....	18
Figure 18 — W3Schools reference page used during Sprint 1 for HTML, CSS and JavaScript syntax	18
Figure 19 — Jira ticket SDABS-6 (View Appointments) showing full task description and acceptance criteria.....	19
Figure 20 — Jira ticket SDABS-7 (Quick Booking Access) showing task details and Done status	19
Figure 21 — Jira ticket SDABS-8 (Home Page and Notifications Panel) showing task details and Done status.....	20
Figure 22 — Jira ticket SDABS-9 (Profile Management) showing task details and Done status	20

Self-Appraisal Form

Student Number	2462286	Name	Sirjeet Dahal
Project	Doctor Appointment Booking System	Date	18th April 2026
Role	Developer	Team	Sigma
Sprint	1	Module	5CS024

Personal Objectives — Performance Measurement

The following table presents the objectives agreed for Sprint 1 of the Doctor Appointment Booking System project, the evidence produced in satisfaction of each objective, and the self-assessed score against the agreed criteria. Cross-references to the supporting appendices and underlying Jira tickets on the SDABS board are provided throughout.

Objectives	Evidence Provided	Evaluation (Student / Tutor)
Choosing Relevant Technologies <i>Tutor feedback:</i>	The project adopted HTML, CSS (Tailwind), JavaScript for the frontend and PHP with MySQL for the backend. Full justification and risk analysis are documented in Appendix A.	9/10 /10
Objective 1 — Home Page & Notifications Panel (SDABS-8) <i>Tutor feedback:</i>	Designed and built the patient home page with dashboard layout, welcome banner, appointment summary cards, and real-time notification panel. Evidence in Appendix B.	9/10 /10
Objective 2 — View Appointments (SDABS-6) <i>Tutor feedback:</i>	Built the appointments view page displaying upcoming, completed, and missed appointments with categorised status and date/time/location details. Evidence in Appendix B.	9/10 /10
Objective 3 — Quick Booking Access (SDABS-7) <i>Tutor feedback:</i>	Implemented a prominent Book Appointment button on the patient dashboard that triggers the booking flow. Evidence in Appendix B.	8/10 /10
Objective 4 — Profile Management (SDABS-9) <i>Tutor feedback:</i>	Developed the patient profile page allowing users to view and edit personal information including name, contact, blood group, address and emergency contact. Evidence in Appendix B.	9/10 /10
Collaboration <i>Tutor feedback:</i>	Actively participated in sprint planning, stand-ups, and team meetings.	13/15 /15

	Maintained clean commit history and collaborated through GitHub. Evidence in Appendices C, D, and E.	
Sprint Retrospective <i>Tutor feedback:</i>	Contributed to the Sprint 1 retrospective with a balanced analysis of wins, pain points, and actionable improvements for Sprint 2. Evidence in Appendix F.	4/5 /5
Total		61/70 /70

Collaboration Development

Collaboration in Sprint 1 was structured deliberately rather than left to chance. Team Sigma followed the Scrum cadence of sprint planning, daily stand-ups, sprint review, and retrospective, with Jira as the source of truth for scope and in-person meetings as the primary channel for synchronous discussion.

Good Communication and File Sharing

Synchronous team communication was conducted through regular in-person meetings where each member reported progress against their Jira tickets, discussed blockers, and planned the day ahead. File sharing for the codebase was managed through Git, with a shared remote repository hosted on GitHub and a feature-branch workflow that isolated work-in-progress from the protected main branch. The team maintained a consistent meeting schedule ensuring blockers were surfaced and resolved promptly.

Evidence for communication and file-sharing practice is presented in Appendix C.

Continuing Personal Development (CPD)

During Sprint 1 I deliberately invested in practical skill growth across four key areas: HTML and Tailwind CSS component design, JavaScript DOM manipulation and Fetch API calls, PHP backend routing and session management, and Git branching and pull-request etiquette. Self-directed learning was structured around short sessions tied to concrete Jira tickets, which translated more directly into delivery quality. Learning resources and a reflection on skill growth are captured in Appendix D.

Issue Tracking

Issues surfaced during Sprint 1 were recorded in Jira as tickets linked back to the originating story, rather than being fixed silently. The SDABS board served as the single source of truth for all work items, ensuring that any defects were visible to the team and could be tracked to closure in a structured manner. Evidence is provided in Appendix E.

Work to Deadlines

All four tickets assigned to me during Sprint 1 — SDABS-6, SDABS-7, SDABS-8, and SDABS-9 — were delivered and moved to the Done column before the sprint review. Delivery was managed by decomposing each Jira story into small, verifiable units of work, logging progress during stand-up, and self-escalating whenever a ticket looked like it might miss the sprint boundary.

Appendix A: Choosing Relevant Technologies

This appendix records the technologies selected for the Doctor Appointment Booking System, the reasoning behind each choice, and the known risks and limitations. The selection was agreed collectively in the team kick-off meeting and recorded on the Jira project board.

Technologies Used

- Programming Languages — HTML5 and CSS3 (with Tailwind CSS) for the frontend; JavaScript (ES6+) for client-side interactivity; PHP 8 for backend API development.
- Database — MySQL / MariaDB as the primary relational store, accessed through PHP MySQLi.
- Version Control — Git, hosted on a shared GitHub remote repository with a feature-branch workflow and pull requests.
- Issue Tracking & Sprint Management — Atlassian Jira (Team Sigma project, SDABS board).
- Communication — In-person team meetings for synchronous stand-ups and sprint ceremonies.
- IDE — Visual Studio Code for frontend and backend development.
- Local Development Server — XAMPP providing Apache, MySQL, and PHP on localhost.

Justification for Chosen Technologies

HTML and Tailwind CSS were selected for the frontend because Tailwind's utility-first approach accelerates UI development while maintaining consistent design across all patient-facing pages. JavaScript was chosen for its native browser support and its ability to handle asynchronous API calls through the Fetch API without requiring additional libraries. PHP was selected for the backend because of its straightforward integration with MySQL, its wide adoption in web hosting environments, and its suitability for session-based authentication patterns. MySQL was chosen because its ACID compliance and relational model are a natural fit for the structured appointment and user data that the system manages. Git was chosen for version control on the strength of its distributed model and deep integration with Jira through smart commits. Jira was selected as it is the de facto industry standard for Agile sprint management.

Risks and Limitations

- HTML/Tailwind CSS — Without disciplined component reuse, utility classes can proliferate across pages and become difficult to maintain. The Tailwind JIT compiler must be run continuously during development to regenerate the output stylesheet.
- JavaScript — Client-side logic is visible in the browser and must never contain sensitive business rules or credentials. Asynchronous code requires careful error handling to avoid silent failures.

- PHP — Plain PHP lacks a built-in routing layer, so URL management must be handled manually. Password storage and session handling require explicit attention to avoid common security vulnerabilities.
- MySQL — Raw write throughput can degrade under concurrent load without proper indexing. Schema changes require careful migration planning to avoid data loss.
- Git — Force-pushes to shared branches and carelessly resolved merge conflicts can destroy work. The team mitigated these risks by protecting the main branch and requiring pull-request reviews before merging.

Appendix B: Implementing Functional Requirements

This appendix evidences the functional deliverables produced during Sprint 1, organised by the four objectives. Every item is traceable to a concrete Jira ticket on the SDABS board and was marked Done before the sprint review. All code was commented to explain key sections, as required by the module specification. Screen captures are referenced by figure number.

Objective 1 — Home Page & Notifications Panel (SDABS-8)

The patient home page was the first functional slice delivered in the sprint as it serves as the primary entry point for all other patient features. The page was implemented as a full dashboard layout featuring a personalised greeting, a welcome banner with today's booking summary (Today Bookings, Upcoming, Completed), a Today Status section showing the current day's appointment details, a mini-calendar widget, a Quick Summary panel, and a real-time notifications panel accessible via a bell icon. The notification panel fetches unread notifications from the PHP backend and displays them with timestamps and a Mark All as Read action.

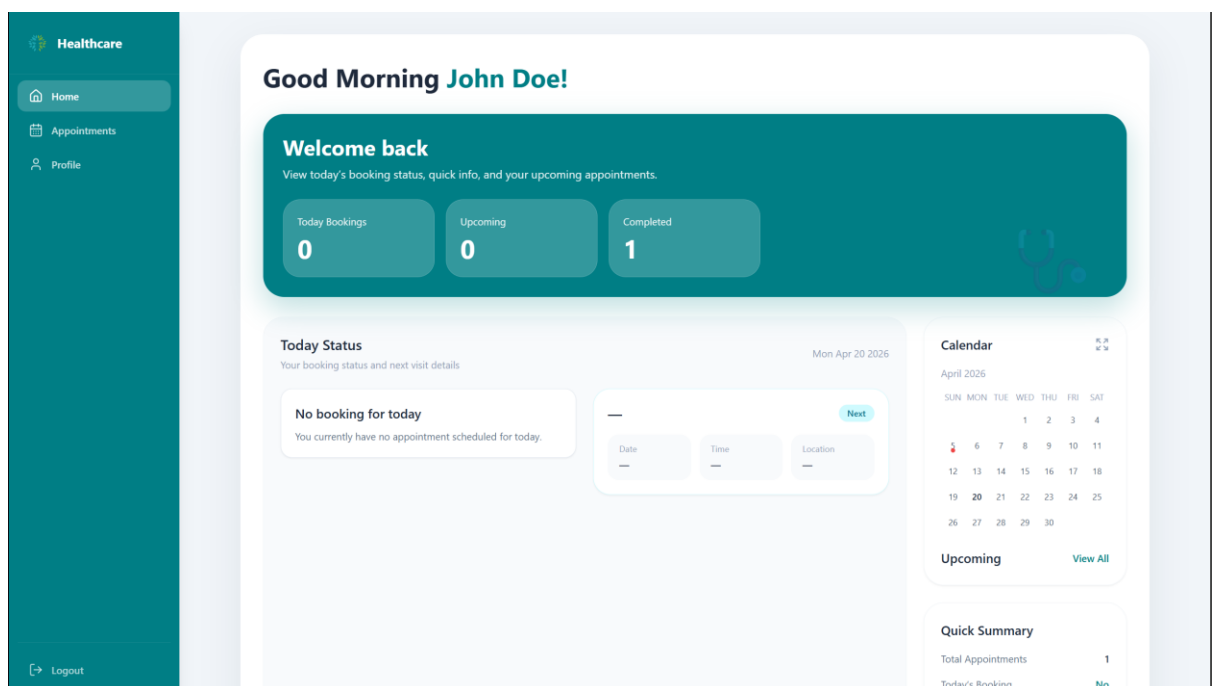


Figure 1 — Screenshot of the patient home page rendered in the browser (SDABS-8)

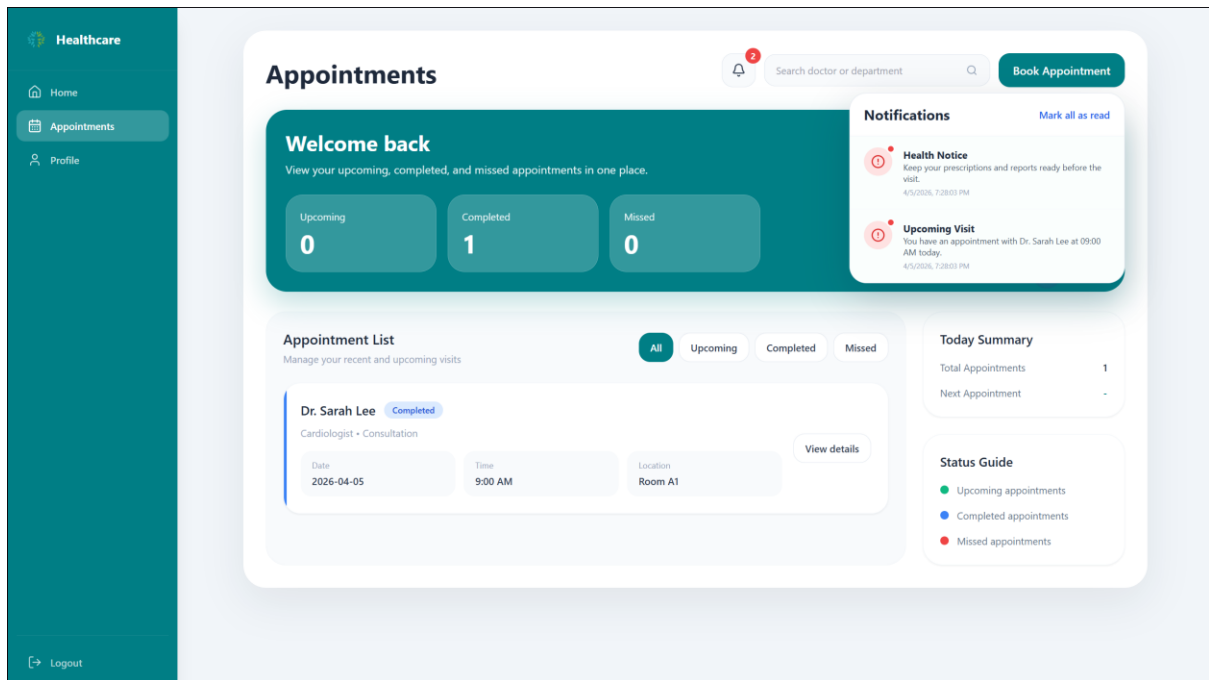


Figure 2 — Screenshot of the notifications panel on the patient page (SDABS-8)

```
// Fetch home page data from the API (bookings count, upcoming, completed, notifications)
async function loadHomeData() {
  const r = await fetch(`${API_BASE}/patient/home.php`, { credentials: "include" });
  const j = await r.json();
  // Throw error if API response is not successful
  if (j.status !== "success") throw new Error(j.message || "Home load failed");
  return j.data;
}

// Fetch all notifications for the logged-in patient from the API
async function loadNotifications() {
  const r = await fetch(`${API_BASE}/patient/notifications.php`, {
    credentials: "include",
  });
  const j = await r.json();
  // Return empty array if fetch fails
  if (j.status !== "success") return [];
  return j.data || [];
}
```

Figure 3 — Screenshot of homepage.js in VS Code showing commented notification fetch and render logic (SDABS-8)

Objective 2 — View Appointments (SDABS-6)

The appointments view page allows patients to see all their appointments categorised by status: Upcoming, Completed, and Missed. The page displays appointment cards with the doctor's name, specialisation, appointment date, time, room number, and a status badge. Filter tabs (All, Upcoming, Completed, Missed) allow the patient to narrow the list, and a search bar supports filtering by doctor name or department. JavaScript fetches the patient's

appointment data from the PHP API and dynamically renders the appointment list with colour-coded status badges.

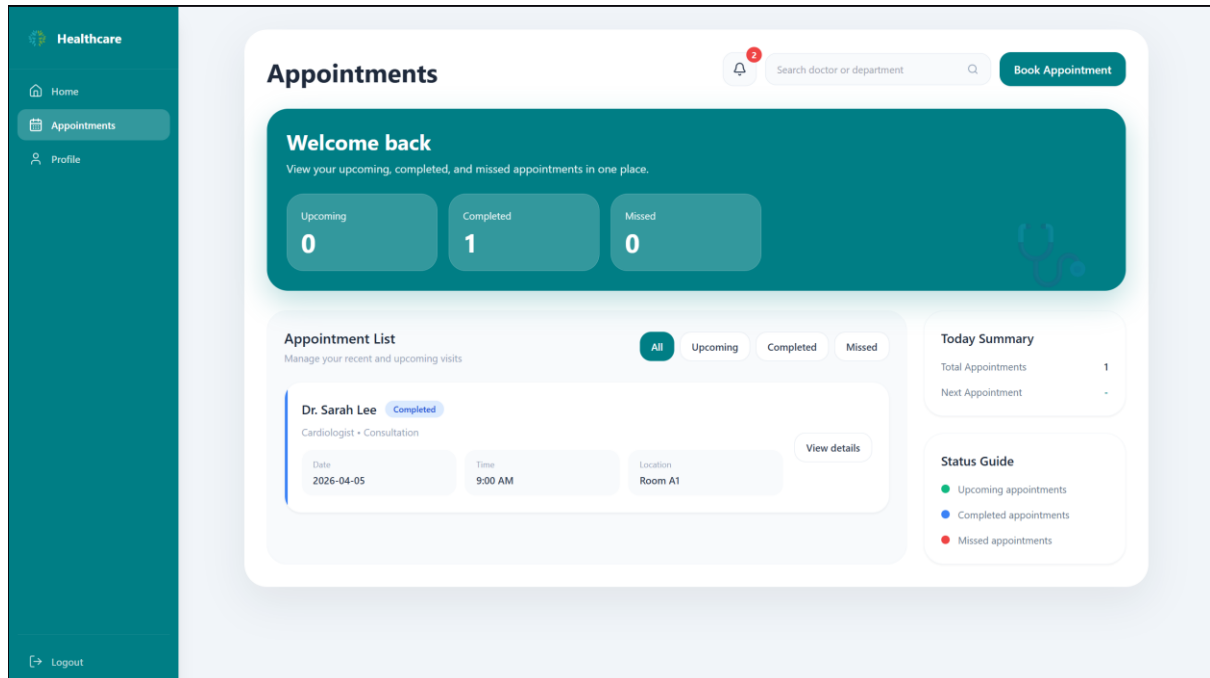


Figure 4 — Screenshot of the View Appointments page in the browser (SDABS-6)

```

// Fetch appointments from the API with optional status filter and search query
async function fetchAppointments() {
  const params = new URLSearchParams();
  // Apply active filter if not showing all
  if (activeFilter !== "all") params.set("status", activeFilter);
  // Apply search query if present
  if (searchQuery) params.set("q", searchQuery);
  const r = await fetch(
    `${API_BASE}/patient/appointments.php?${params.toString()}`,
    { credentials: "include" },
  );
  const j = await r.json();
  // Throw error if API response is not successful
  if (j.status !== "success") throw new Error(j.message || "Load failed");
  return j.data || [];
}

// Update appointment count summary cards in the UI
function renderCounts(rows) {
  const upcoming = rows.filter((a) => a.status_key === "upcoming").length;
  const completed = rows.filter((a) => a.status_key === "completed").length;
  const missed = rows.filter((a) => a.status_key === "missed").length;

  // Update count display elements
  document.getElementById("upcomingCount").textContent = upcoming;
  document.getElementById("completedCount").textContent = completed;
  document.getElementById("missedCount").textContent = missed;
  document.getElementById("totalCount").textContent = rows.length;

  // Display the next upcoming appointment date and time
  const next = rows.find((a) => a.status_key === "upcoming");
  document.getElementById("nextAppointment").textContent = next
    ? `${next.app_date} • ${formatTime12h(next.app_time)}`
    : "-";
}

```

Figure 5 — Screenshot of dashboard.js in VS Code showing commented appointment fetch and render logic (SDABS-6)

Objective 3 — Quick Booking Access (SDABS-7)

The Quick Booking Access feature provides patients with a prominent, always-visible Book Appointment button on the appointments dashboard page. The button is positioned in the top-right area of the page header and redirects the patient into the booking flow. A notification bell icon sits alongside the button, displaying an unread count badge when notifications are pending. This satisfies the acceptance criteria of making the booking call-to-action visible and accessible without navigating away from the appointments view.

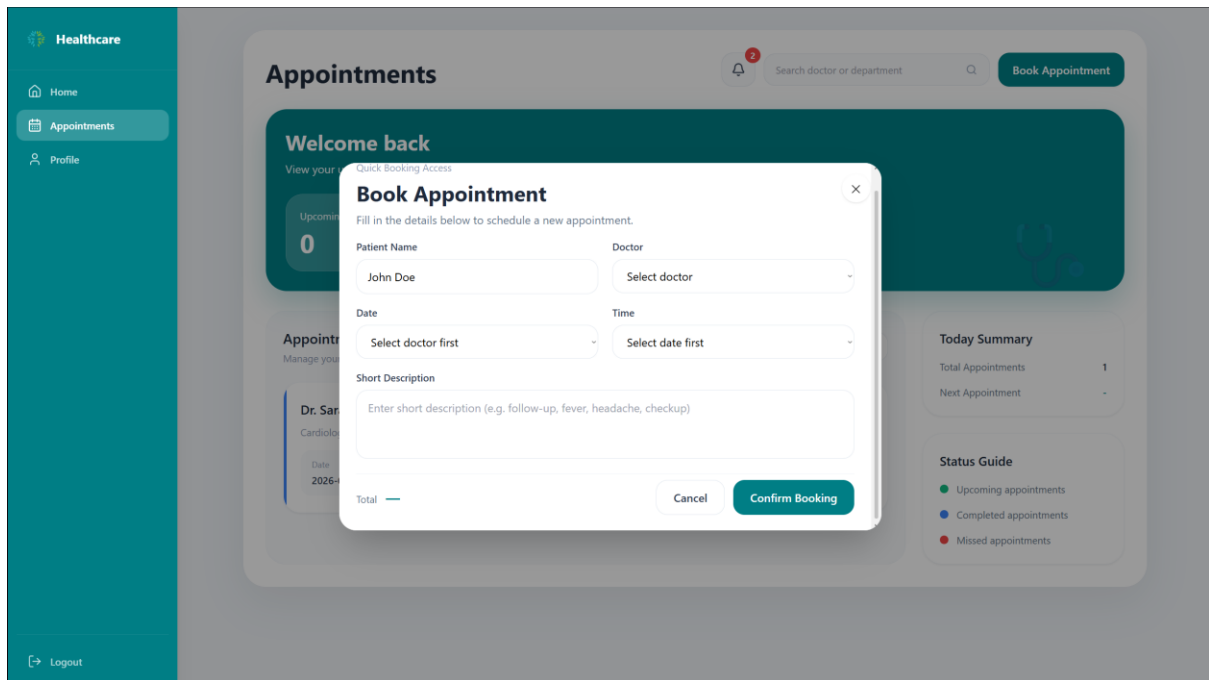


Figure 6 — Screenshot of the Quick Booking Access button on the patient dashboard (SDABS-7)

```
// Fetch available doctors from the API and populate the doctor dropdown
async function loadDoctors() {
  const r = await fetch(`${API_BASE}/patient/doctors.php`, {
    credentials: "include",
  });
  const j = await r.json();
  // Throw error if API response is not successful
  if (j.status !== "success") throw new Error(j.message || "Doctors failed");
  doctorsList = j.data || [];
  const sel = document.getElementById("doctor");
  sel.innerHTML = '<option value="">Select doctor</option>';
  // Add each doctor as an option in the dropdown
  doctorsList.forEach((d) => {
    const opt = document.createElement("option");
    opt.value = d.doctor_id;
    opt.textContent = `${d.full_name} (${d.specialization})`;
    sel.appendChild(opt);
  });
}
```

Figure 7 — Screenshot of booking.js in VS Code showing loadDoctors() function fetching available doctors from the API code (SDABS-7)

Objective 4 — Profile Management (SDABS-9)

The profile management page allows authenticated patients to view and edit their personal information. The page displays a profile card showing the patient's full name, email, personal information (phone number, date of birth, age, gender, blood group), and address and

emergency contact details. An Edit Profile button opens an editable form that submits updates to the PHP backend, which persists the changes to the patient's database record. JavaScript handles the toggle between view and edit modes, form validation, and the PUT request to the profile update endpoint.

The screenshot displays a web application interface for 'Healthcare'. The main header shows 'Profile Management' with the date 'Mon Apr 20 2026' and an 'Edit Profile' button. A sidebar on the left contains navigation links: Home, Appointments, Profile (selected), and Logout. The main content area features a profile card for 'John Doe' with the email 'john@email.com'. The card is divided into two sections: 'Personal Information' and 'Address & Emergency Contact'. The 'Personal Information' section includes fields for Full Name, Email, Phone Number, Date of Birth, Age, Gender, and Blood Group. The 'Address & Emergency Contact' section includes fields for Address, Emergency Contact Name, and Emergency Contact Phone.

Personal Information	
FULL NAME	John Doe
EMAIL	john@email.com
PHONE NUMBER	9801112220
DATE OF BIRTH	1990-05-15
AGE	-
GENDER	-
BLOOD GROUP	O+

Address & Emergency Contact	
ADDRESS	Kathmandu, Nepal
EMERGENCY CONTACT NAME	-
EMERGENCY CONTACT PHONE	-

Figure 8 — Screenshot of the Profile Management page in the browser (SDABS-9)

```

// Fetch the patient profile data from the API
async function loadProfile() {
  try {
    console.log("Loading profile from:", `${API_BASE}/patient/profile.php`);

    // Set a 10 second timeout to abort the request if it takes too long
    const controller = new AbortController();
    const timeout = setTimeout(() => controller.abort(), 10000);

    // Fetch profile data from the API
    const response = await fetch(`${API_BASE}/patient/profile.php`, {
      credentials: "include",
      signal: controller.signal
    });

    clearTimeout(timeout);
    console.log("Response status:", response.status);

    // Fall back to mock data if response is not OK
    if (!response.ok) {
      console.warn("Response not OK:", response.statusText);
      loadMockProfile();
      return;
    }

    const result = await response.json();
    console.log("Response data:", result);

    // Fall back to mock data if API returns an error status
    if (result.status !== "success") {
      console.warn("API Error:", result.message);
      loadMockProfile();
      return;
    }

    // Fall back to mock data if no profile data is returned
    if (!result.data) {
      console.warn("No data in response");
      loadMockProfile();
      return;
    }

    // Store profile data and update the view mode display
    patientProfile = result.data;
    updateViewMode();
    document.getElementById("current-date").textContent = new Date().toLocaleDateString();
    console.log("Profile loaded successfully:", patientProfile);
  } catch (error) {
    console.error("Error loading profile:", error);
    console.warn("Falling back to mock data...");
    // Fall back to mock data if an exception occurs
    loadMockProfile();
  }
}

// Save the updated profile data to the API
async function saveProfile() {
  // Validate form before submitting
  if (!validateForm()) {
    return;
  }
}

```

Figure 9 — Screenshot of profile.js in VS Code showing commented profile fetch and update logic (SDABS-9)

Use of Version Control

Git was used throughout the sprint to coordinate work across team members and to preserve an auditable history of every change. The team followed a feature-branch workflow: each Jira ticket was developed on a dedicated branch, reviewed via pull request, and merged into the main branch only after approval. All code was committed regularly to GitHub, providing a clear record of individual contributions.

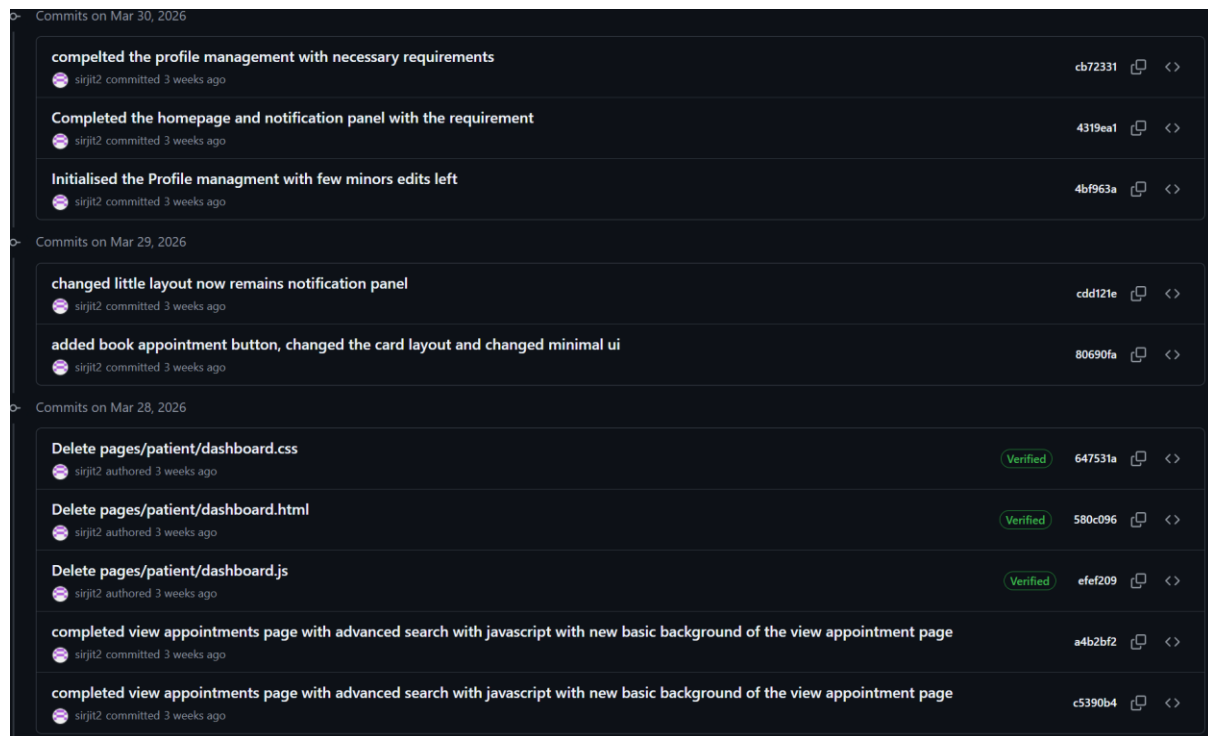


Figure 10 — Screenshot of GitHub commit history showing Sprint 1 commits by myself (page 1)

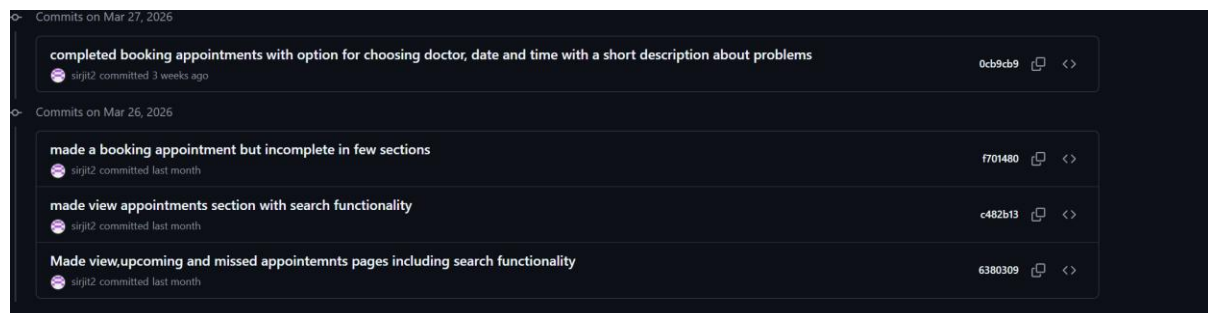


Figure 11 — Screenshot of GitHub commit history showing Sprint 1 commits by sirjit2 (page 2)



Figure 12 — Screenshot of GitHub contributor graph showing sirjit2 commit activity during Sprint 1

Appendix C: Evidence of Good Communication and File Sharing

Team communication was conducted through regular in-person meetings where progress, blockers, and plans were discussed openly. File sharing for code was handled through a shared Git remote repository on GitHub. The figures below evidence the communication patterns that sustained the sprint.



Figure 13 — Team Sigma in-person meeting during Sprint 1

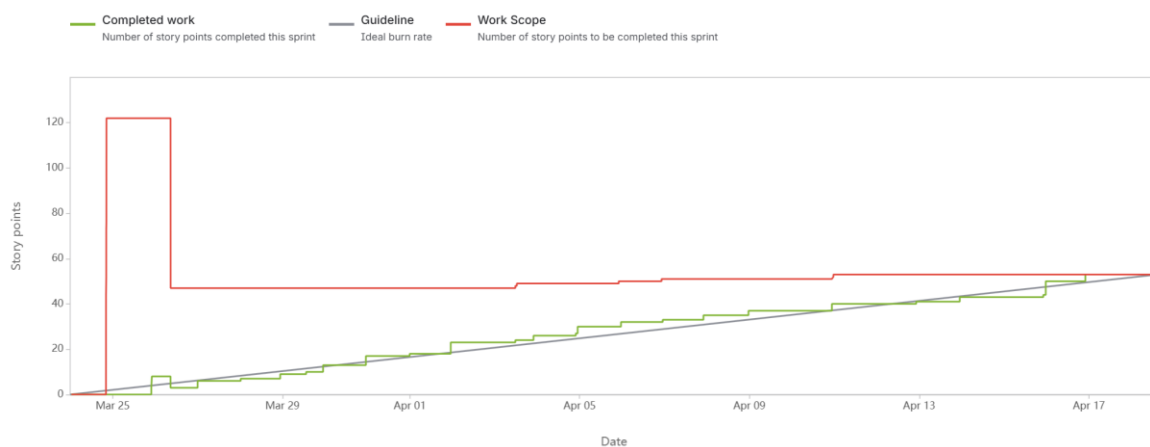


Figure 14 — Jira Sprint 1 completion report for Team Sigma (SDABS board)

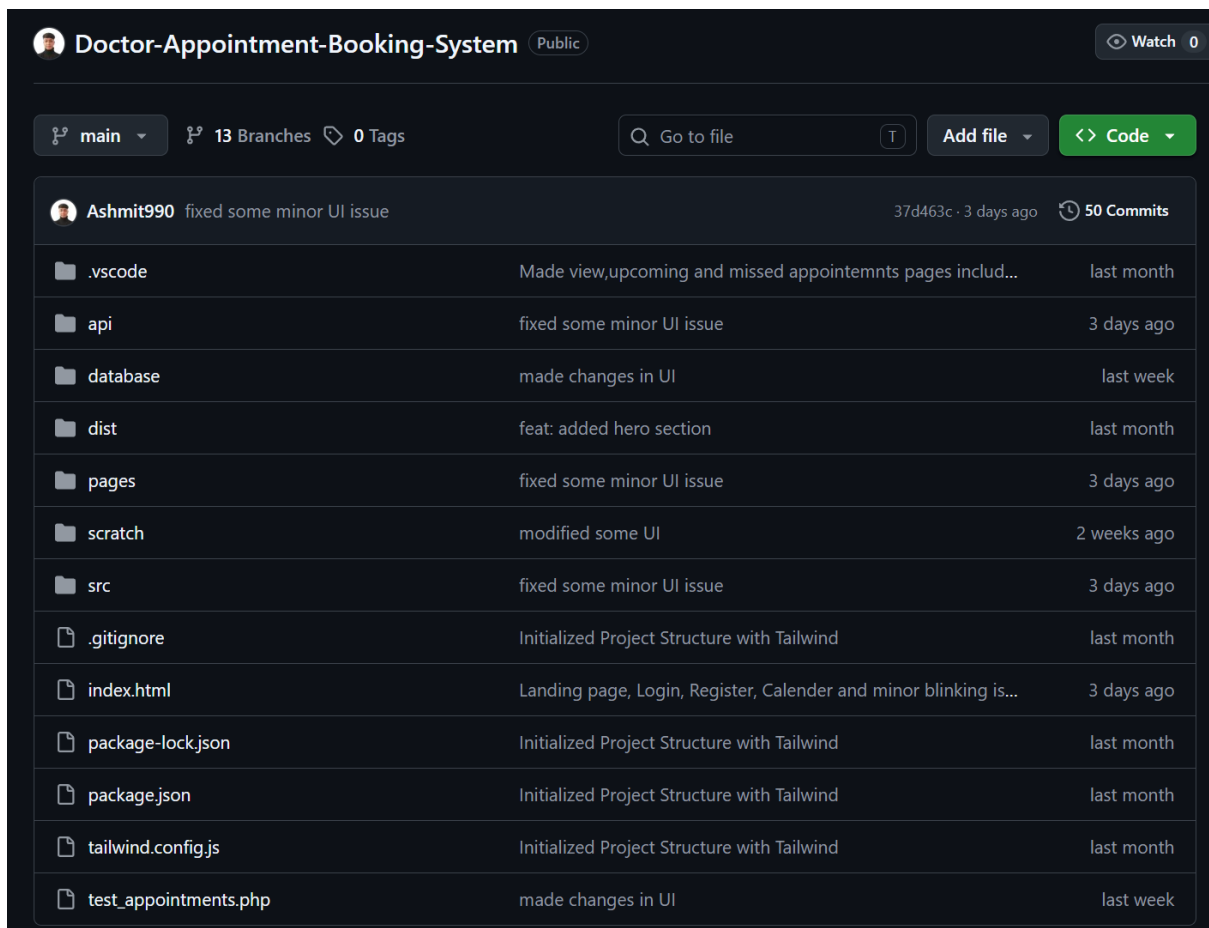


Figure 15 — GitHub repository showing Team Sigma project with branches, commits, and contributors

Appendix D: Continuing Personal Development (CPD)

Sprint 1 provided a deliberate opportunity to close specific skill gaps identified at the start of the module. Three areas received the most focused attention: Tailwind CSS component design and responsive layout, JavaScript Fetch API and asynchronous DOM manipulation, and PHP session management and MySQL query writing. Rather than consuming broad tutorial content, each learning session was paired directly with the Jira ticket being worked on, so that concepts were applied within the same day and reinforced through immediate practice.

Frontend skills progressed most visibly. At the start of the sprint my experience with Tailwind CSS was limited to basic utility classes; by the end of the sprint I was composing responsive multi-column layouts, building interactive notification panels, and managing dynamic state through JavaScript without a framework. Backend skills also developed significantly, as integrating PHP session-based authentication with the patient profile and appointment APIs required a clear understanding of server-side request handling and JSON response formatting.

Agile Scrum itself formed the fourth axis of CPD. Prior to this module I had only encountered Scrum academically; Sprint 1 exposed me to the lived experience of planning a backlog, breaking down stories into tasks, attending stand-ups with discipline, and participating in a retrospective that produced concrete actions for Sprint 2. The primary external resources that supported this growth are cited below.

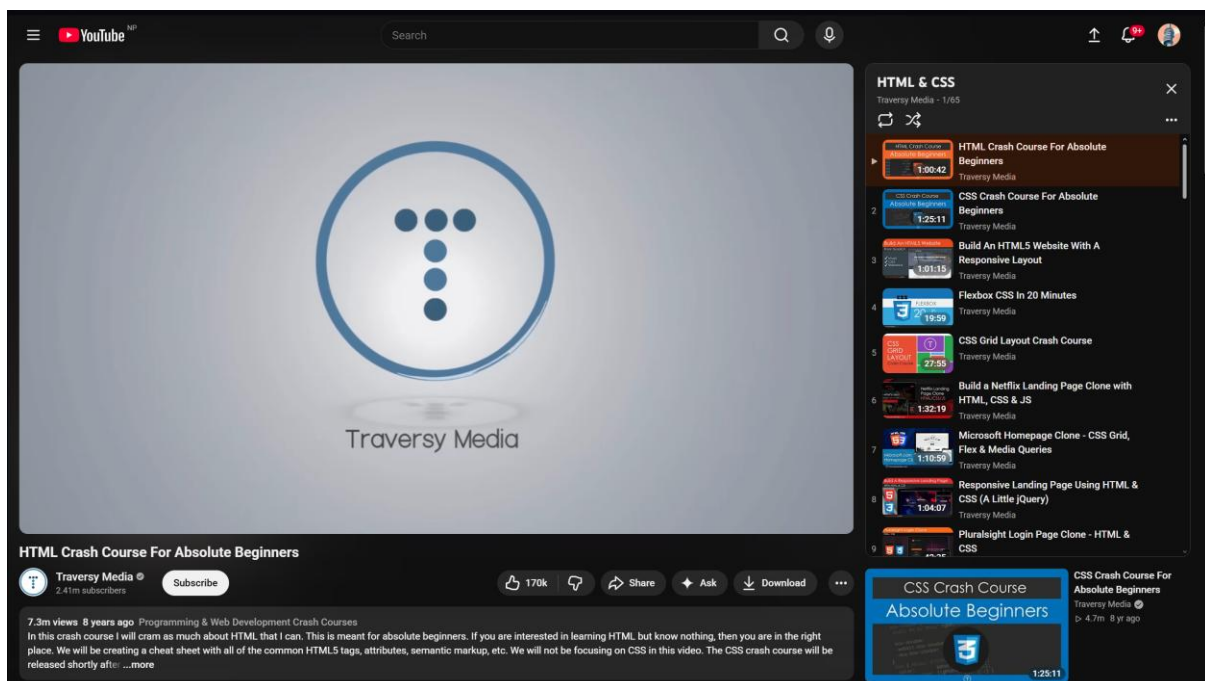


Figure 16 — Traversy Media YouTube tutorial used during Sprint 1 for HTML, CSS and JavaScript reference



Figure 17 — The Net Ninja YouTube tutorial used during Sprint 1 for PHP and MySQL reference

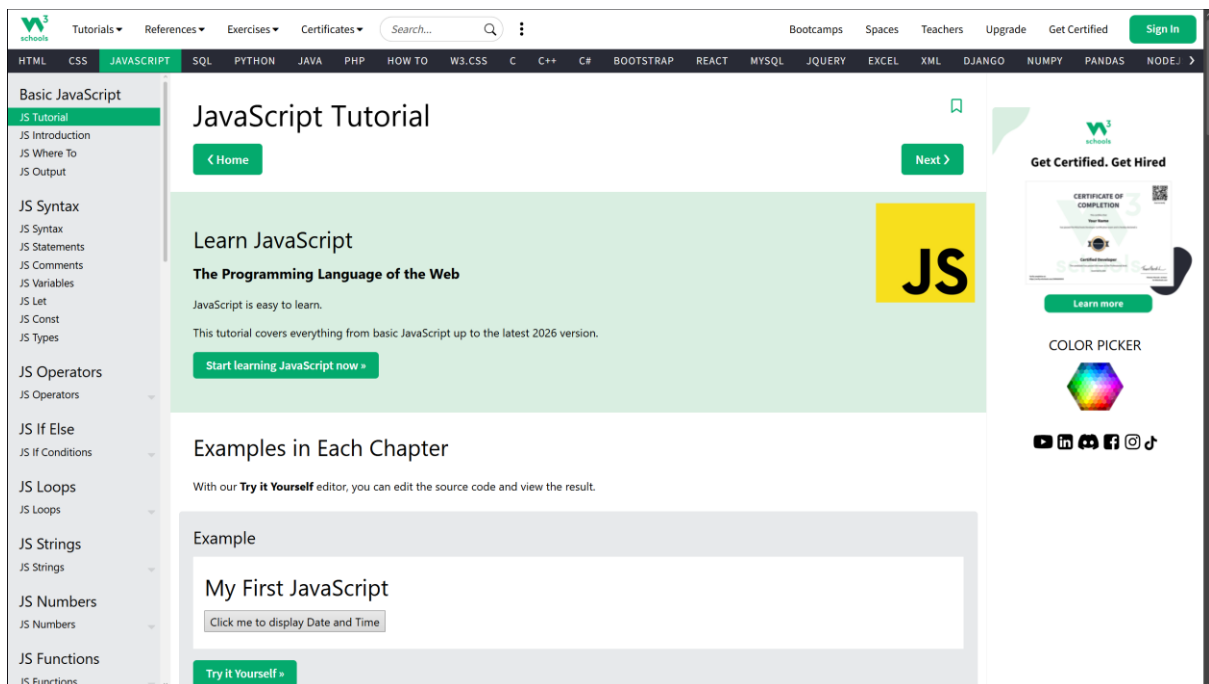


Figure 18 — W3Schools reference page used during Sprint 1 for HTML, CSS and JavaScript syntax

Appendix E: Evidence of Issue Tracking

During Sprint 1 all work items were recorded and tracked in Jira as tickets on the SDABS board. Each ticket contained a clear description, acceptance criteria, and task breakdown. The board served as the single source of truth ensuring that all deliverables were visible to the team and could be tracked from To Do through to Done in a structured manner.

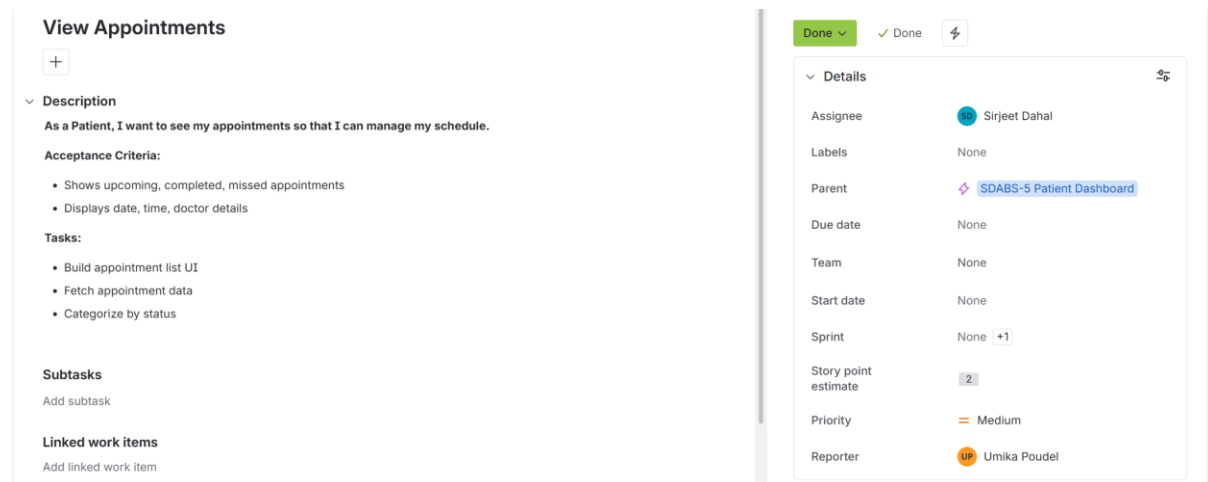


Figure 19 — Jira ticket SDABS-6 (View Appointments) showing full task description and acceptance criteria

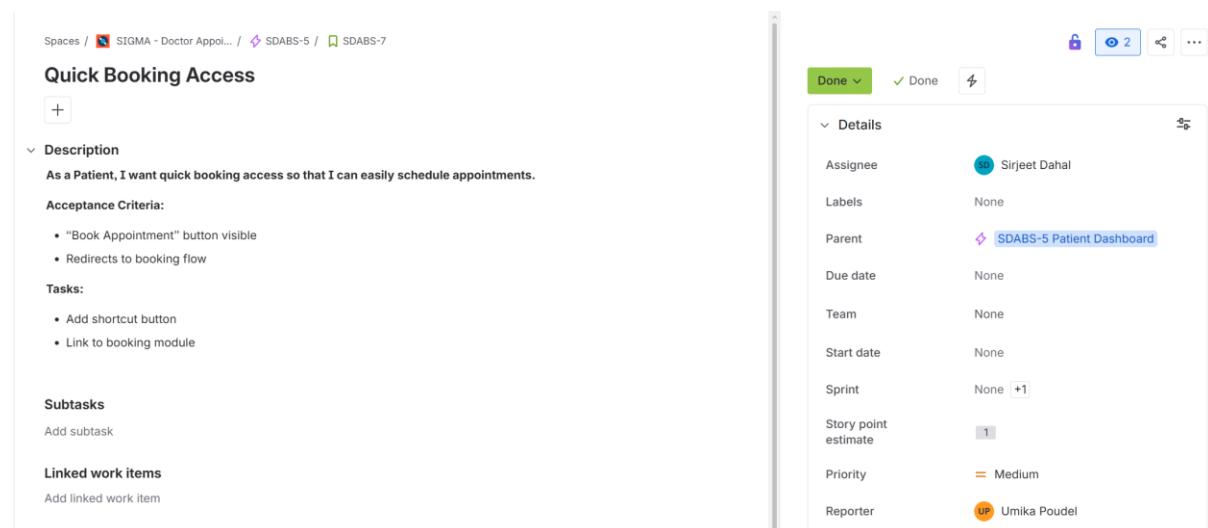


Figure 20 — Jira ticket SDABS-7 (Quick Booking Access) showing task details and Done status

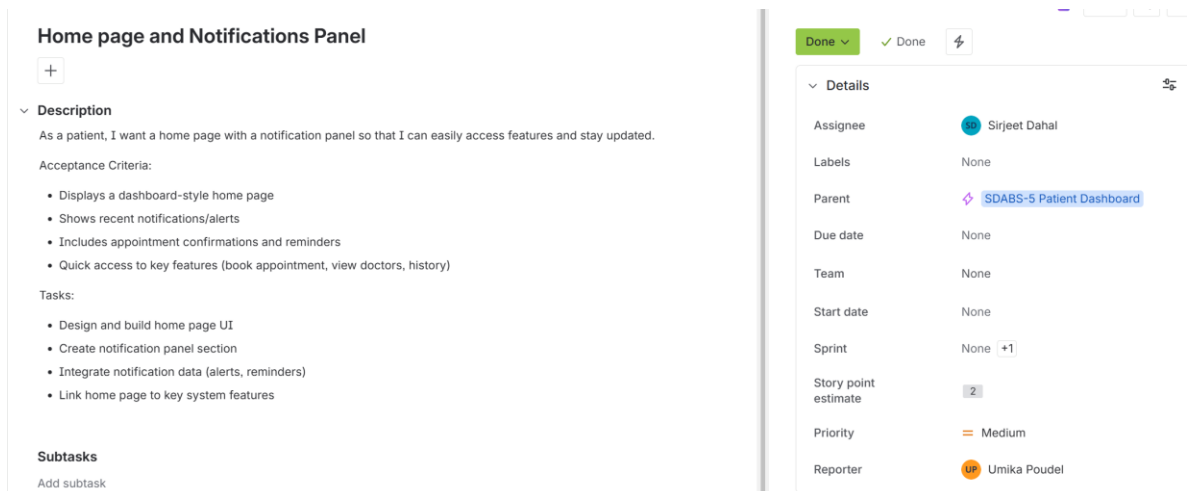


Figure 21 — Jira ticket SDABS-8 (Home Page and Notifications Panel) showing task details and Done status

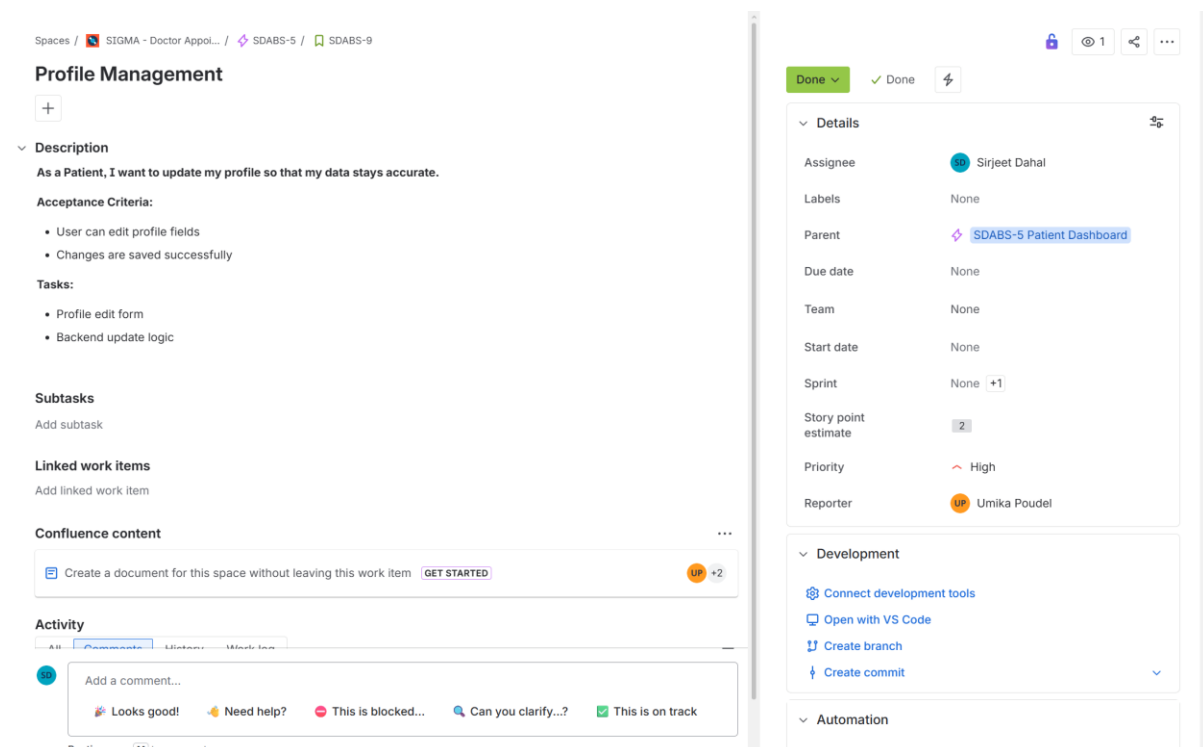


Figure 22 — Jira ticket SDABS-9 (Profile Management) showing task details and Done status

Appendix F: Sprint Retrospective

Sprint 1 closed with a structured retrospective facilitated by the team. The discussion was organised around four questions: what went well, what did not go as planned, what was learned, and what will change in Sprint 2. The subsections below capture the findings, contextualised to my own deliverables on the Doctor Appointment Booking System.

What Went Well

- All four Jira tickets assigned to me (SDABS-6, SDABS-7, SDABS-8, SDABS-9) were completed, reviewed, and moved to Done before the sprint review.

- Regular in-person meetings kept the team aligned and allowed blockers to be surfaced and resolved quickly.
- The feature-branch and pull-request workflow on GitHub prevented breakage on the main branch and enforced a review culture from day one.
- Using Tailwind CSS from the start ensured visual consistency across all patient-facing pages without writing custom CSS from scratch.
- Integration between the PHP backend and the JavaScript frontend was smoother than expected due to clear API contracts agreed during sprint planning.

Challenges Faced

- Integrating the notification panel with the PHP backend required careful handling of JSON responses and asynchronous timing, which introduced debugging overhead mid-sprint.
- Estimating the profile management story (SDABS-9) proved difficult because the edit-mode toggle and form validation added scope not fully visible during refinement.
- Cross-browser layout inconsistencies in the appointments view required additional CSS adjustments not accounted for in the original estimate.

Lessons Learned

- API contracts between frontend and backend should be documented explicitly before implementation begins, not discovered during integration.
- Edit/view toggle patterns for profile pages are worth treating as a design decision at story-refinement time, not during implementation.
- Short, ticket-anchored learning sessions are more effective than open-ended tutorial watching for sprint-time skill growth.
- Pull-request review is cheap insurance against regressions that would otherwise consume disproportionate debugging time.

Future Improvements for Sprint 2

- Introduce a more formalised stand-up structure with a written record of Yesterday / Today / Blockers to improve visibility.
- Agree a team-wide definition of done that includes cross-browser smoke testing for all customer-facing screens.
- Begin Sprint 2 with an explicit design spike for each story estimated at more than three story points.
- Add input sanitisation and server-side validation to all form endpoints to reduce security risk as the application grows.

Overall, Sprint 1 established a robust functional foundation for the patient-facing side of the Doctor Appointment Booking System and established the team practices that will make subsequent sprints more efficient to deliver. The retrospective outcomes have been logged in Jira as Sprint 2 preparation tasks so that the improvements carry forward.

Appendix G: Risk Register

This appendix presents the risk register maintained for the Doctor Appointment Booking System project across Sprint 1 and the remainder of the module window. Risks are grouped into three categories — technical, schedule, and team — and are recorded with a likelihood and impact score on a one-to-five scale, yielding a composite risk score that drives the mitigation priority. The register is a living document: each risk is reviewed at the start of sprint planning, status is updated at the retrospective, and new risks identified during the sprint are added with a fresh identifier rather than overwriting the previous entry.

Scoring Scheme

Each risk is scored on two axes. Likelihood captures how probable the risk is to occur within the remaining sprint window given current evidence; impact captures the consequence to the Doctor Appointment Booking System MVP if the risk does occur. The composite score (Likelihood $\times$ Impact) ranges from 1 (trivial) to 25 (critical). Risks scoring 12 or higher are reviewed in every stand-up until the score falls, risks between 6 and 11 are reviewed weekly, and risks below 6 are re-evaluated at the end of the sprint.

Score	Rating	Interpretation
1	Very Low	Likelihood: will almost certainly not occur / Impact: negligible effect on sprint delivery.
2	Low	Likelihood: unlikely but plausible / Impact: minor inconvenience, absorbed within sprint.
3	Medium	Likelihood: could occur in this sprint / Impact: one ticket slips or requires rework.
4	High	Likelihood: expected unless actively managed / Impact: sprint goal partially missed.
5	Very High	Likelihood: near certainty / Impact: sprint goal missed or MVP compromised.

Risk Register

The register below enumerates the fifteen risks currently tracked for the project. Status values are Open (identified but no mitigation executed), Planned (mitigation scheduled but not yet active), Mitigating (mitigation in progress), and Mitigated (residual risk considered acceptable). Ownership follows the Scrum Team role: the named owner is accountable for reporting status at stand-up, not necessarily for single-handed remediation.

ID	Category	Risk Description	L $\times$ I = Score	Mitigation Strategy	Owner	Status
R01	Technical	Plain-text password storage in the MySQL users table exposes patient credentials if the database is accessed directly or backed up insecurely.	4 $\times$ 5 = 20	Replace plain-text storage with PHP password_hash (bcrypt) on registration and password_verify on login in Sprint 2; add this as the first Sprint 2 ticket on the SDABS board.	Backend Developer	Planned
R02	Technical	Doctor approval bypassed if the admin approval workflow contains a missing role-check, allowing a newly	3 $\times$ 5 = 15	Require server-side approval status check on every protected PHP endpoint; include a test case in the definition of	Backend Developer	Open

ID	Category	Risk Description	L×I = Score	Mitigation Strategy	Owner	Status
		registered doctor to see patients before being approved.		done for any ticket touching doctor registration or role management.		
R03	Technical	Double-booking risk: a patient submitting the booking form twice (double-click or slow connection) could create duplicate appointments for the same slot.	3×4 = 12	Disable the Confirm Booking button on first click and re-enable only on error; add a server-side uniqueness check on avail_id per patient before inserting the appointment row.	Developer (Booking)	Mitigating
R04	Technical	XAMPP MySQL service failing to start on a team member's local machine blocks development and testing of all appointment and profile API endpoints.	4×3 = 12	Document the port-conflict fix (netstat, taskkill, my.ini port change) in the team wiki; ensure all developers can connect to a shared dev database on the team lead's machine as a fallback.	All Developers	Mitigating
R05	Technical	Cross-browser rendering inconsistencies in the Tailwind-styled notification panel or appointment cards if developers test only in Chrome.	4×2 = 8	Add a cross-browser smoke-test step to the definition of done; rotate review duty so each pull request is checked in at least one non-Chrome browser before merge.	Developer (Frontend)	Mitigating
R06	Technical	PHP session hijacking or session fixation if session IDs are not regenerated after login, exposing authenticated patient accounts.	2×4 = 8	Call session_regenerate_id(true) immediately after successful login; set session cookie flags (HttpOnly, Secure, SameSite=Strict) in the PHP configuration.	Backend Developer	Open
R07	Technical	Force-push or accidental rebase on a shared GitHub branch destroys another developer's in-flight work, causing data loss and wasted effort.	2×4 = 8	Enforce branch protection on main (no force-push, require PR and one approval); document the Git branching workflow in the team wiki; reinforce during in-person stand-ups.	Team Lead	Mitigated
R08	Schedule	Story estimation error causes a ticket (e.g., SDABS-7 Quick Booking Access) to expand mid-sprint as the booking flow proves more complex than anticipated.	4×3 = 12	Introduce design spikes for stories above three points; track estimate-vs-actual per ticket and recalibrate at each sprint planning; deliver the highest-value slice first if scope expands.	Scrum Master	Planned
R09	Schedule	Coursework deadlines from other modules overlapping with Sprint 2 compress individual capacity and risk	4×3 = 12	Each developer declares external module load at sprint planning; team velocity is reduced accordingly rather than overcommitting; identify	All Developers	Open

ID	Category	Risk Description	L×I = Score	Mitigation Strategy	Owner	Status
		incomplete ticket delivery.		buffer stories that can slip without breaking the MVP.		
R10	Schedule	Scope creep from sprint review feedback inflates the Sprint 2 backlog and threatens MVP delivery by the module submission deadline.	3×4 = 12	Hold a scope-freeze checkpoint at the midpoint of each sprint; any new request goes to the product backlog and is triaged at the next planning session rather than added mid-sprint.	Product Owner	Open
R11	Team	A team member becomes unavailable due to illness, exam pressure, or personal obligations during Sprint 2, leaving a critical patient-dashboard ticket stranded.	3×4 = 12	Pair-programme on high-risk tickets so at least two team members understand each feature area; keep branches small and pushed frequently so work is always recoverable.	Scrum Master	Open
R12	Team	Knowledge silos form around the PHP backend and the notification panel because only one developer authored each subsystem, creating single points of failure.	4×3 = 12	Schedule 15-minute internal knowledge-share sessions at the end of each sprint; require the non-author to review at least one PR per story; document key design decisions in the team wiki.	Team Lead	Mitigating
R13	Team	Communication breakdown between frontend and backend developers (e.g., PHP API response shape changes not announced) causes wasted rework on dependent patient-side pages.	3×3 = 9	Maintain an API contract document in the repository updated in the same commit as any endpoint change; announce breaking changes at the in-person stand-up before the PR is merged.	All Developers	Mitigating
R14	Team	Uneven contribution (free-rider risk) damages team morale and leaves high performers absorbing additional load late in the sprint.	2×4 = 8	Make Jira ticket assignments and GitHub commit counts visible to all; Scrum Master flags silent members within 48 hours; tutor is notified if the pattern persists into Sprint 2.	Scrum Master	Open
R15	Team	Tailwind CSS JIT compilation not running during active development causes stale styles in the browser, producing UI inconsistencies that waste debugging time.	2×2 = 4	Document the npx tailwindcss --watch command in the project README; include it in the onboarding checklist for any new team member joining the project.	Developer (Frontend)	Mitigated

Priority Risks and Rationale

Four risks warrant an explicit narrative because they combine high impact with a plausible path to occurrence in Sprint 2. R01 (plain-text password storage) scores twenty — the highest in the register — because a compromised database backup would expose every

patient's credentials in readable form. The mitigation is straightforward (PHP password_hash and password_verify) and has been logged as the first Sprint 2 ticket precisely because it must be fixed before the system handles real users. R02 (doctor approval bypass) scores fifteen because a single missing server-side role check could allow an unapproved doctor to access patient data through a direct API call; the mitigation requires a mandatory approval-status check on every protected PHP endpoint, enforced through the pull-request definition of done. R03 (double-booking) scores twelve and is already partially mitigated by the client-side button disable; the residual risk is a rapid second submission from two separate browser tabs, which requires the planned server-side uniqueness constraint on the availability slot. R08 (estimation error) and R09 (external module deadlines) together address the primary schedule threat for Sprint 2: underestimating story complexity compounded by academic load. The proposed design-spike discipline and explicit capacity declaration at sprint planning are deliberately lightweight so they survive the pressure of a busy semester rather than being abandoned under workload.

Review Cadence

The register is revisited at three ceremonies. At sprint planning, every open risk is reviewed and a decision is made on whether the planned mitigation work should be pulled into the upcoming sprint backlog as a concrete Jira ticket. At the daily stand-up, any risk scoring twelve or higher is reported against by its owner, even if the update is simply that status is unchanged. At the retrospective, the register is updated with new risks surfaced during the sprint, closed risks are moved to a dedicated archive sheet so that their mitigation trail is preserved, and the scoring of remaining risks is re-assessed in light of what was actually observed during the sprint. This cadence prevents the register from becoming a static document that is produced once and ignored, which is the most common failure mode of risk management on student projects.

References

- Schwaber, K. and Sutherland, J. (2020) The Scrum Guide. Available at: <https://scrumguides.org/scrum-guide.html>.
- W3Schools (2026) HTML, CSS and JavaScript Reference. Available at: <https://www.w3schools.com>.
- Traversy Media (2026) HTML, CSS, JavaScript & PHP Web Development Tutorials [YouTube Channel]. Available at: <https://www.youtube.com/@TraversyMedia>.
- The Net Ninja (2026) HTML & CSS, JavaScript, PHP & MySQL Tutorials [YouTube Channel]. Available at: <https://www.youtube.com/@NetNinja>.
- PHP Group (2026) PHP Manual. Available at: <https://www.php.net/manual>.
- Atlassian (2026) Jira Software Cloud Documentation. Available at: <https://support.atlassian.com/jira-software-cloud/>.
- Tailwind CSS (2026) Tailwind CSS Documentation. Available at: <https://tailwindcss.com/docs>.
- Chacon, S. and Straub, B. (2014) Pro Git. 2nd edn. Apress. Available at: <https://git-scm.com/book/en/v2>.
- Mozilla Developer Network (2026) Fetch API Documentation. Available at: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API.